JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Interface ClassFileTransformer

`public interface ClassFileTransformer`

A transformer of class files. An agent registers an implementation of this interface using the `addTransformer` method so that the transformer's `transform` method is invoked when classes are loaded, `redefined`, or `retransformed`. The implementation should override one of the `transform` methods defined here. Transformers are invoked before the class is defined by the Java virtual machine.

There are two kinds of transformers, determined by the `canRetransform` parameter of `Instrumentation.addTransformer(ClassFileTransformer,boolean)`:

- *retransformation capable* transformers that were added with `canRetransform` as true
- *retransformation incapable* transformers that were added with `canRetransform` as false or where added with `Instrumentation.addTransformer(ClassFileTransformer)`

Once a transformer has been registered with `addTransformer`, the transformer will be called for every new class definition and every class redefinition. Retransformation capable transformers will also be called on every class retransformation. The request for a new class definition is made with `ClassLoader.defineClass` or its native equivalents. The request for a class redefinition is made with `Instrumentation.redefineClasses` or its native equivalents. The request for a class retransformation is made with `Instrumentation.retransformClasses` or its native equivalents. The transformer is called during the processing of the request, before the class file bytes have been verified or applied. When there are multiple transformers, transformations are composed by chaining the `transform` calls. That is, the byte array returned by one call to `transform` becomes the input (via the `classfileBuffer` parameter) to the next call.

Transformations are applied in the following order:

- Retransformation incapable transformers
- Retransformation incapable native transformers
- Retransformation capable transformers
- Retransformation capable native transformers

For retransformations, the retransformation incapable transformers are not called, instead the result of the previous transformation is reused. In all other cases, this method is called. Within each of these groupings, transformers are

called in the order registered. Native transformers are provided by the `ClassFileLoadHook` event in the Java Virtual Machine Tool Interface).

The input (via the `classfileBuffer` parameter) to the first transformer is:

- for new class definition, the bytes passed to `ClassLoader.defineClass`
- for class redefinition, `definitions.getDefinitionClassFile()` where `definitions` is the parameter to `Instrumentation.redefineClasses`
- for class retransformation, the bytes passed to the new class definition or, if redefined, the last redefinition, with all transformations made by retransformation incapable transformers reapplied automatically and unaltered; for details see `Instrumentation.retransformClasses`

If the implementing method determines that no transformations are needed, it should return `null`. Otherwise, it should create a new `byte[]` array, copy the input `classfileBuffer` into it, along with all desired transformations, and return the new array. The input `classfileBuffer` must not be modified.

In the retransform and redefine cases, the transformer must support the redefinition semantics: if a class that the transformer changed during initial definition is later retransformed or redefined, the transformer must insure that the second class output class file is a legal redefinition of the first output class file.

If the transformer throws an exception (which it doesn't catch), subsequent transformers will still be called and the load, redefine or retransform will still be attempted. Thus, throwing an exception has the same effect as returning `null`. To prevent unexpected behavior when unchecked exceptions are generated in transformer code, a transformer can catch `Throwable`. If the transformer believes the `classFileBuffer` does not represent a validly formatted class file, it should throw an `IllegalClassFormatException`; while this has the same effect as returning null. it facilitates the logging or debugging of format corruptions.

Note the term *class file* is used as defined in chapter 4 The `class` File Format of *The Java Virtual Machine Specification*, to mean a sequence of bytes in class file format, whether or not they reside in a file.

API Note:

> Great care must be taken when transforming core JDK classes which are at the same time required during the transformation process as this can lead to class circularity or linkage errors.
>
> If for example the invocation of `transform()` for a class `C` requires loading or resolving the same class `C`, an error is thrown that is an instance of `LinkageError` (or a subclass). If the `LinkageError` occurs during reference resolution (see section §5.4.3 Resolution of *The Java Virtual Machine*

Specification) for a class `D`, the resolution of the corresponding reference in class `D` will permanently fail with the same error at any subsequent attempt. This means that a `LinkageError` triggered during transformation of `C` in a class `D` not directly related to `C` can repeatedly occur later in arbitrary user code which uses `D`.

Since:

1.5

See Also:

`Instrumentation`

Method Summary

All Methods | Instance Methods

Modifier and Type	Method	Description
default byte[]	**transform**(ClassLoader loader, String className, Class<? > classBeingRedefined, ProtectionDomain protect byte[] classfileBuffer)	Transforms the given class file and returns a new replacement class file.
default byte[]	**transform**(Module module, ClassLoader loader, String className, Class<? > classBeingRedefined, ProtectionDomain protect byte[] classfileBuffer)	Transforms the given class file and returns a new replacement class file.

Method Details

transform

```
default byte[] transform(ClassLoader loader,
                         String className,
                         Class<?> classBeingRedefined,
                         ProtectionDomain protectionDomain,
                         byte[] classfileBuffer)
                  throws IllegalClassFormatException
```

Transforms the given class file and returns a new replacement class file. This method is invoked when the `Module` bearing `transform` is not overridden.

Implementation Requirements:

The default implementation returns null.

Parameters:

`loader` - the defining loader of the class to be transformed, may be `null` if the bootstrap loader

`className` - the name of the class in the internal form of fully qualified class and interface names as defined in *The Java Virtual Machine Specification*. For example, `"java/util/List"`.

`classBeingRedefined` - if this is triggered by a redefine or retransform, the class being redefined or retransformed; if this is a class load, `null`

`protectionDomain` - the protection domain of the class being defined or redefined

`classfileBuffer` - the input byte buffer in class file format - must not be modified

Returns:

a well-formed class file buffer (the result of the transform), or `null` if no transform is performed

Throws:

`IllegalClassFormatException` - if the input does not represent a well-formed class file

transform

```
default byte[] transform(Module module,
                         ClassLoader loader,
                         String className,
                         Class<?> classBeingRedefined,
                         ProtectionDomain protectionDomain,
                         byte[] classfileBuffer)
                  throws IllegalClassFormatException
```

Transforms the given class file and returns a new replacement class file.

Implementation Requirements:

The default implementation of this method invokes the `transform` method.

Parameters:

`module` - the module of the class to be transformed

`loader` - the defining loader of the class to be transformed, may be `null` if the bootstrap loader

`className` - the name of the class in the internal form of fully qualified class and interface names as defined in *The Java Virtual Machine Specification*. For example, `"java/util/List"`.

`classBeingRedefined` - if this is triggered by a redefine or retransform, the class being redefined or retransformed; if this is a class load, `null`

`protectionDomain` - the protection domain of the class being defined or redefined

`classfileBuffer` - the input byte buffer in class file format - must not be modified

Returns:

a well-formed class file buffer (the result of the transform), or `null` if no transform is performed

Throws:

`IllegalClassFormatException` - if the input does not represent a well-formed class file

Since:

9

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.